

Product Requirements Document (PRD): Full-Stack & AI Learning Management System (LMS)

Goal: To deliver a highly engaging and effective learning experience that produces portfolio-ready technical talent using a Project-Based Learning (PBL) model, granular tracking, and advanced AI integration.

1. The Core Learning Path (Content Consumption & Practice)

This section covers the direct student interface with course material (Units, Sections, Sub-Sections). **The order and number of Sub-Sections are dynamically determined by the CRM.**

Element	Detailed Feature of Each Section	Tech Stack for Each Section	User Flow Overview
Learning Sub-Section	Content Delivery: Must support written content, embedded videos, and external links. Tracking: Implements Video Tracking Level (start/stop/pause, drop-off) and Scroll Level tracking (minimum view threshold for completion). Engagement: Integrates a Mandatory Reflection Stage prompt. Progress Persistence:	Frontend: React/Next.js for component structure and dynamic loading. Data: Custom Event Bus/Segment for capturing granular event data.	Student Flow: 1. Navigates to a Sub-Section. 2. Consumes content (video/text), being tracked passively. 3. Completes the reflection prompt to proceed.

	<i>Completion status must be persisted to the backend database.</i>		
Quiz Sub-Section	Functionality: Supports Small Quizzes (Multiple Choice, True/False) with automated Answer Evaluation and Scoring. Tracking: Logs all interaction via Button Level tracking for UX analysis. Streaks: Successful completion contributes to the daily learning Streaks Feature. Progress Persistence: Quiz scores and attempt history must be saved to the backend.	Frontend: React/Next.js for quiz interface. Backend: Node.js/FastAPI handles scoring logic and updates the analytics database.	Student Flow: 1. Attempts the quiz questions. 2. Submits answers (button clicks tracked). 3. Receives immediate score and feedback. 4. Streaks status is updated.
Short Code Assignment Sub-Section	Embedded IDE: Fully functional, in-browser editor environment (Sandbox). Environment: Supports File System and Real-Time Execution for MERN Stack and Python/FastAPIs. Assessment:	Embedded IDE: Monaco Editor / VS Code API. Execution: Containerization (Docker/Kubernetes) is MANDATORY for isolated, ephemeral environments. AI Service: Gemini API for LLM-Based Evaluation.	Student Flow: 1. Codes solution in the IDE. 2. Clicks Run/Submit. 3. Code executes in a secure container. 4. Receives instant, AI-enhanced feedback and score.

	Automated Grading via unit tests and LLM-Based Evaluation for code style and efficiency. Streaks: Successful submission contributes to the Streaks Feature. Progress Persistence: <i>Code submission, test results, and final score must be saved to the backend.</i>		
Long-Format Assignment Sub-Section	Submission: Acceptance of project files or live deployment link, with preference for GitHub Link Submission. Scoring: Supports Scoring in Progress by human mentors/evaluators. Version Control: Mandates following Git workflows (Version Control Simulation). Tracking: Detailed Submissions Level logging of attempts, time taken, and score history. Progress Persistence: <i>All</i>	Backend: Node.js/Python manages submission metadata and handles real-time score updates. Real-time: WebSockets (e.g., Socket.IO) for score progress updates.	Student Flow: 1. Works on the project locally. 2. Submits the GitHub repository link. 3. System logs submission details. 4. Receives periodic mentor feedback until final score is allocated.

	<i>submission metadata, scores, and mentor feedback must be saved to the backend.</i>		
--	---	--	--

2. Code Execution & AI Services (Technical Infrastructure)

This defines the specialized services required for hands-on practice and smart assistance.

Element	Detailed Feature of Each Section	Tech Stack for Each Section	User Flow Overview
Code Execution Sandbox	Isolation: Uses isolated, ephemeral containers per user/session for secure execution. Support: Handles compilation and execution for MERN, FastAPI, Python environments. Provisioning: Includes Isolated DB Instances (MongoDB, PostgreSQL) accessible by the running code.	Execution: Containerization (Docker/Kubernetes). Database: Isolated MongoDB/PostgreSQL instances per session.	System Flow: 1. User starts assignment. 2. Platform spins up a dedicated container. 3. Container mounts the assignment files and database instance. 4. Container is destroyed upon session end.
AI Doubt Clearing	Contextual Chatbot: Provides help by feeding the LLM the exact content (Unit/Section/Sub-	AI Service: Gemini API for advanced LLM reasoning and contextual ingestion. Data: MongoDB/PostgreSQL	Student Flow: 1. Student asks a question in the chat interface. 2. The system sends the question + current

	Section, and optionally student's code) the student is currently viewing. Interaction: Supports conversational flow for troubleshooting and conceptual explanation.	QL for storing conversation history.	course context to the AI. 3. AI responds with a contextualized answer.
Authentication & User Identity	Secure Login: Must use Google OAuth 2.0 for primary user login. Persistence: The resulting user ID must be the primary key for <i>all</i> progress data, analytics, and personalization features. Data Security: Ensure compliance with Google's authentication standards.	Auth: Firebase Auth / Custom OAuth Integration. Backend: Node.js/Python server manages token validation and user session creation.	User Flow: 1. User clicks "Sign In." 2. Redirects to Google login page. 3. Upon successful authentication, returns to the LMS, where a user session is established using the validated Google ID.

3. User Engagement & Support (Motivation & Help)

These are cross-cutting features designed to motivate students and provide external support.

Element	Detailed Feature of Each Section	Tech Stack for Each Section	User Flow Overview
Notifications	Alert Types: Provides alerts for class schedules, assignment deadlines,	Backend: Node.js/Python notification service. Real-time: WebSockets for	Student Flow: 1. A deadline is approaching or a mentor scores a project. 2. Student

	submission feedback, and system events. Channels: Supports email, in-app, and push notifications.	in-app alerts.	receives an immediate in-app notification and an email alert.
Mentorship Setup	Structured Booking: Students must select the specific Unit, Section, and Sub-Section relevant to their doubt. Context: Ability to add a text prompt, image, or link for context during booking.	Frontend: React/Next.js booking interface. Backend: Node.js/Python scheduling and calendar integration.	Student Flow: 1. Student identifies a roadblock. 2. Uses the booking system to specify the exact course context. 3. Schedules a session with a mentor.
Student Profile & Leaderboards	Profile: Displays comprehensive view of persisted progress, streaks status, quiz scores, long assignment history, and achieved badges/certifications. Leaderboards: Motivational ranking based on scores, completion rates, and streaks.	Frontend: React/Next.js dynamic profile components. Data: Data Warehouse (e.g., Google BigQuery) performs aggregation and ranking queries.	Student Flow: 1. Student clicks on their profile icon. 2. Views a personalized dashboard of their performance and ranking against peers.

4. Administration & Analytics (Platform Management)

Features accessible only to internal administrators for content lifecycle management and business intelligence.

Element	Detailed Feature of Each Section	Tech Stack for Each Section	User Flow Overview
Content Management	Dynamic Course Builder: Centralized interface to manage, edit, publish, and archive <i>all</i> content pieces. CRITICAL: Must support dynamically assigning and sequencing Sub-Sections (Learning, Quiz, Short Code, Long Format) within any given Section. Database Model: Each Sub-Section must be stored with a type (e.g., 'quiz', 'learning') and a sequence_index for ordering.	Frontend: React/Next.js dashboard UI (Desktop/Laptop optimized). Backend: Node.js/Python managing CRUD operations on core course structure data (MongoDB/Postgre SQL with focus on flexible schema to support dynamic content).	Admin Flow: 1. Admin logs in to the CRM. 2. Selects a Unit/Section. 3. Uses the Dynamic Course Builder tool to add and reorder Sub-Sections before publishing.
Reporting Dashboard & Granular Data Access	Visualization: Dynamic dashboard visualizing all tracked data. Analysis: Capabilities across College Level, Student Level, and Gender Level. Raw Access: Provides direct, raw access to all granular data	Data Warehouse: Google BigQuery (or similar) is mandatory to ingest high-volume, granular event data and power complex analytical queries.	Admin Flow: 1. Admin accesses the Reporting Dashboard. 2. Selects filters (e.g., "College X," "Gender: Female"). 3. Runs a query to analyze drop-off points (Video Tracking Level data) for that segment. 5. Admin clicks "Raw Data

	(Scroll Level, Button Level, Video Tracking Level, Exercise Level, Submissions Level). <i>Data Persistence: All granular tracking data and student progress (scores, attempts, completion) must be permanently stored in the Data Warehouse/Analytics DB.</i>		Export" to download the underlying Video Tracking Level events for that filtered segment.
--	---	--	--

5. Visual Layout and Detailed User Flow (UX/UI)

This section defines the intended visual structure and expanded user interaction details for key components. **Note:** These descriptions require companion artifacts (wireframes and flowcharts) to be generated and maintained alongside the PRD.

5.1. Core Learning Path Layouts

Element	Visual Layout Description	Expanded User Flow (UX/UI)
Learning Sub-Section	Responsive Three-Column Layout: 1. Left Nav (Course Outline): Fixed, collapsible. 2. Main Content (70%): Written content, embedded video players. 3. Right Sidebar (30%): Dynamic progress bar, table of contents, and Streaks status widget. A sticky footer contains the "Mark Complete" button	1. User clicks the "Next" button in the Left Nav. 2. Content loads, tracking starts immediately. 3. After 80% scroll depth is recorded, the "Mark Complete" button highlights. 4. Clicking "Mark Complete" triggers the mandatory Reflection modal, preventing accidental progression.

	and triggers the Reflection prompt modal.	
Short Code Assignment	Tri-Pane Integrated Development Environment (IDE): 1. Instructions (25%): Top-left panel displaying problem statement and unit test expectations. 2. Code Editor (50%): Middle panel using Monaco Editor for coding. 3. Console/Output (25%): Bottom panel showing real-time execution output, test results, and Isolated DB Status . A dedicated button group (Run, Test, Submit) is fixed above the Code Editor.	1. User codes solution in the middle pane. 2. Clicks "Run." The Console panel flashes "Running..." and displays container output. 3. Clicks "Submit." A modal appears showing the LLM-Based Evaluation score, detailed suggestions on code efficiency, and style points.
Long-Format Assignment	Content/Submission Split: Standard content consumption layout, but the main area ends with a large, dedicated submission card. This card includes the GitHub Link text field, a "Version Control Check" button, and a scrollable log of previous submission attempts and Scoring in Progress updates from the mentor.	1. User pastes GitHub URL. 2. Clicks "Version Control Check" (System confirms Git workflow compliance). 3. Clicks "Final Submit." 4. The Submission Log instantly updates to show "Pending Review." 5. If a mentor updates the "Scoring in Progress," a real-time notification (via WebSocket) triggers, and the log updates without page refresh.

5.2. AI and Support Layouts

Element	Visual Layout Description	Expanded User Flow (UX/UI)

AI Doubt Clearing	Floating Chat Widget: A small, circular, collapsible icon docked to the bottom-right corner of the screen. When expanded, it is a vertical chat interface. The chat header prominently displays the context (e.g., "AI Context: React Hooks (Unit 3, Sec 2)").	1. User clicks the floating icon. 2. The chat panel opens. 3. The system automatically inserts the context header. 4. User types a question, and the chat simulates low-latency, real-time responses from the Gemini API .
Mentorship Setup	Guided Booking Modal: A multi-step modal overlay focused on capturing essential context first. Step 1: Select Unit/Section/Sub-Section (mandatory hierarchical dropdowns). Step 2: Text prompt and file/link uploads. Step 3: Calendar/Mentor selection.	1. User clicks "Book Mentor." 2. The modal forces context selection, preventing vague bookings. 3. After context is set, the booking form and calendar appear. 4. Confirmation screen displays the selected context and time to the user.
Student Profile & Leaderboards	Gamified Dashboard: Profile is a card-based view focusing on visual metrics: a prominent Streaks counter (e.g., fire icon), badge gallery, and a mini-chart showing score trends. Leaderboards are a clean, tabular view with clear filtering options (Score, Completion Rate, Streaks).	1. User clicks the profile avatar. 2. The Streaks card is the first element seen. 3. User navigates to the Leaderboard tab. 4. Filters the table by "Top 10% in Unit 5" using a dropdown, and the table updates dynamically.

5.3. Administration Layouts

Element	Visual Layout Description	Expanded User Flow (UX/UI)
---------	---------------------------	----------------------------

Content Management	CRM Interface (Desktop Optimized): Left-side navigation features a flexible tree-view structure mirroring the course hierarchy. The main area is a complex form/editor panel supporting different input types based on the selected content piece. Dynamic Sequencing UI: For a selected Section, the main panel features an ordered list of existing Sub-Sections with drag-and-drop handles for reordering. An "Add New Sub-Section" button triggers a modal requiring the Admin to select the content Type ('learning', 'quiz', 'short_code', 'long_format') before loading the specific content creation form.	1. Admin selects a specific Section in the tree-view. 2. The Dynamic Sequencing UI loads, showing the current Sub-Section order (e.g., Quiz $\rightarrow$ Learning $\rightarrow$ Short Code). 3. Admin uses drag-and-drop to reorder them (e.g., Learning $\rightarrow$ Quiz $\rightarrow$ Short Code). 4. Admin clicks "Add New Sub-Section," selects 'quiz' from a dropdown, and is presented with the Quiz creation form. 5. Admin clicks "Publish Live" to save the new order and content structure.
Reporting Dashboard	BI Visualization Tool: Uses a fixed left-side menu for report navigation and filtering (e.g., filter by College, Gender, Date Range). The main area features interactive charts and tables displaying aggregated data (e.g., drop-off heatmaps, average assignment score per college). A dedicated "Raw Data Export" button is provided for Granular Data Access.	1. Admin logs in and loads the dashboard. 2. Selects "Video Drop-off Analysis." 3. Filters the data by "Gender: Female" and "College: X." 4. The chart updates to show drop-off rates, highlighting specific timestamps. 5. Admin clicks "Raw Data Export" to download the underlying Video Tracking Level events for that filtered segment.